

Sargantana: A 1 GHz+ In-Order RISC-V Processor with SIMD Vector Extensions in 22nm FD-SOI

Víctor Soria-Pardos^{*1}, Max Doblas^{*1}, Guillem López-Paradís^{*}, Gerard Candón^{†2}, Narcís Rodas^{*}, Xavier Carril^{*}, Pau Fontova-Musté^{*}, Neiel Leyva^{*}, Santiago Marco-Sola^{*§}, and Miquel Moretó^{*‡}
[‡]Universitat Politècnica de Catalunya (UPC) [§] Universidad Autònoma de Barcelona (UAB)
^{*}Barcelona Supercomputing Center (BSC) [†] Codasip
{name.surname}@bsc.es gerard.candon@gmail.com

Abstract—The RISC-V open Instruction Set Architecture (ISA) has proven to be a solid alternative to licensed ISAs. In the past 5 years, a plethora of industrial and academic cores and accelerators have been developed implementing this open ISA.

In this paper, we present Sargantana, a 64-bit processor based on RISC-V that implements the RV64G ISA, a subset of the vector instructions extension (RVV 0.7.1), and custom application-specific instructions. Sargantana features a highly optimized 7-stage pipeline implementing out-of-order write-back, register renaming, and a non-blocking memory pipeline. Moreover, Sargantana features a Single Instruction Multiple Data (SIMD) unit that accelerates domain-specific applications. Sargantana achieves a 1.26 GHz frequency in the typical corner, and up to 1.69 GHz in the fast corner using 22nm FD-SOI commercial technology.

As a result, Sargantana delivers a $1.77\times$ higher Instructions Per Cycle (IPC) than our previous 5-stage in-order DVINO core, reaching 2.44 CoreMark/MHz. Our core design delivers comparable or even higher performance than other state-of-the-art academic cores performance under Autobench EEMBC benchmark suite. This way, Sargantana lays the foundations for future RISC-V based core designs able to meet industrial-class performance requirements for scientific, real-time, and high-performance computing applications.

Index Terms—RISC-V, computer architecture, vector instructions, domain-specific accelerators

I. INTRODUCTION

The revolution of open source software was one of the most important achievements in Computer Science history [1]. Nowadays, open source projects, such as Linux and GNU, are the foundations of thousands of open and proprietary software solutions. Recently, Open Instruction Set Architectures (ISA) have been proposed to start an analogous revolution in the hardware design field, creating alternatives to the mainstream monolithic ISAs such as x86, Armv9, or PowerPC.

RISC-V is the most popular open ISA due to its modular design based on extensions [2]. Using RISC-V, computer architects can choose the smallest instruction set that fits their design requirements. Since the release of the open RISC-V ISA, we have experienced an exponential growth industrial and academic processor implementations [3]–[10]. These projects have promoted the RISC-V ISA to the industrial level, boosting the design and development of a whole ecosystem of processors, accelerators, and a full software stack.

Moreover, RISC-V has paved the way for software-hardware co-design through its custom instructions reserved space. As a result, computer architects can freely design specific instructions to accelerate domain-specific applications in multiple areas such as bioinformatics, machine learning, or graph analysis. In particular, commercial frameworks, such as Codasip, offer simple software-hardware co-design opportunities using a high level specification language, which generates Register Transfer Language (RTL) code, providing compiler support [11].

This paper presents Sargantana, a 7-stage in-order core that implements RISC-V's RV64G ISA along with a subset of RVV extension 0.7.1 [12]. Our pipeline implements out-of-order write-back, register renaming, a 128-bit SIMD and supports multiple memory accesses in-flight. Moreover, Sargantana achieves a high clock frequency above 1 GHz in a modern 22nm technology node.

In addition, Sargantana follows a hardware-software co-design philosophy targeting the acceleration of domain-specific applications. Through custom instructions we accelerate the Parallel Bit Myers [13] and Wavefront Alignment Algorithm [14] in computational genomics, or the Classic McEliece cryptosystem used in post-quantum cryptography [15]. Furthermore, the SIMD vector unit's design can efficiently exploit the augmented throughput of reduced length data types.

We have compared Sargantana against other in-order academic cores using Autobench EEMBC [16] and CoreMark [17] benchmark. Sargantana delivers an IPC similar or better than other state-of-the-art academic cores.

The rest of the paper is structured as follows. First, Section II reviews the architecture of other well-known academic RISC-V cores. Next, Section III and V describes the pipeline microarchitecture of Sargantana. Then, Section IV evaluates and analyzes the performance of our model and compares our design against other academic RISC-V cores. Finally, Section VI summarizes the contributions of this work.

II. BACKGROUND

This section briefly reviews the most relevant academic cores, used in Section IV to evaluate the performance of Sargantana. All the listed cores are 64-bit RISC-V compliant and able to boot a Linux distribution.

The Rocket core [18] is a 5-stage in-order core. It has 16 KB instruction cache, a non-blocking data cache, and a 64 KB L2 cache. The Rocket core has been fabricated using

¹These authors contributed equally.

²Contributed to this work prior to joining Codasip.

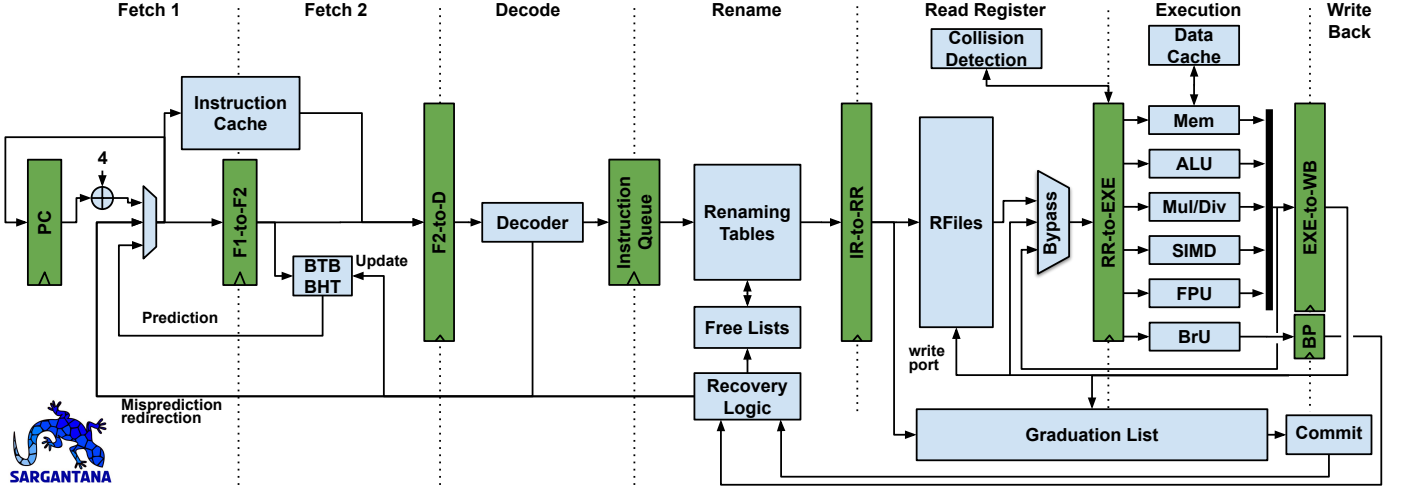


Fig. 1. Block diagram of the 7-stages that compose Sargantana pipeline. Each stage is divided with a vertical dotted line. The graduation list structure is not located at any specific stage. For a detailed description of each pipeline stage and functional unit see Section III.

the TSMC40GPLUS process achieving a maximum maximum frequency of 1.3 GHz when running at high operational voltage.

Risky [19] is another academic in-order core that has two different versions of the pipeline. It has a 5-stage pipeline that achieves 1 GHz on a 22nm technology and a high-performance 7-stage pipeline that achieves 1.4 GHz. Both versions of Risky use a 16 KB instruction and 4-way data caches.

Lastly, Ariane is a 6-stage in-order core that has an out-of-order write-back mechanism to exploit more parallelism with long latency operations. The instruction cache stores 16 KB in 4-ways, while the data cache has 32 KB split in 8-ways. The most relevant tape-out [8] of Ariane was fabricated in the 22nm FD-SOI node from GlobalFoundries. On this tape-out, Ariane was signed-off with a 0.90 GHz worst case, and near to 1.08 GHz on the typical corner.

III. ARCHITECTURE

In this section we describe the 7-stage pipeline implementation of the Sargantana design. Compared to its predecessor, the Sargantana pipeline design improves the performance while makes it more modular. In turn, its modular design and flexibility allows the seemingly addition of different functional units and facilitates the incorporation of further optimizations. Figure 1 presents the block diagram of the full Sargantana pipeline. Our design supports ISA extensions for floating point and SIMD instructions. Moreover, the 7-stage pipeline design has been optimized to target a high frequency over 1 GHz.

A. Front-End Design

The front-end is divided into three different stages. There are two fetch stages plus the decoding stage. There is a zero latency instruction queue between the front-end and the back-end that decouples the fetch of instructions from their execution.

The operations done in each stage are:

- **Fetch Stage 1:** a request with the actual PC address is sent to the first-level cache.
- **Fetch Stage 2:** the response of the first level cache is received. In case of having an instruction cache miss, this

stage will stall until the instruction cache responds. Also, the Branch Target Buffer and Branch History Table [20] are accessed to predict the next PC.

- **Decode Stage:** The decoding of the particular instruction is done in this stage. Besides, JAL instruction redirects the PC since the target is already known. Finally, the resulting decoded instruction is pushed to the instruction queue.

The instruction cache is a custom design based on the lowRISC project instruction cache [21]. The instruction cache only has two cycle access time since it follows a Virtually Indexed and Physically Tagged (VIPT) structure, removing a cycle of latency by doing the TLB access in parallel with the tag array. We have split the access latency to the instruction cache in two different stages. This design decision enables pipelined accesses that increment the peak frequency of the core. This way, the front-end fetches one instruction per cycle if there is no miss in the instruction cache.

B. Back-End Design

In general, a full out-of-order back-end increases the design and verification costs of the pipeline. Meanwhile, an in-order back-end can significantly reduce the performance of the entire core because it blocks the execution of new instruction until long latency instructions finish. Our pipeline design combines strengths from both architectural design, keeping in-order issue and enabling out-of-order write-back of instructions at the same time. This way, our pipeline design enables the execution of non-dependent short-latency instructions in parallel with long latency instructions.

Program order is the foundation of programming models. Therefore, it must be preserved in case of exceptions or interruptions to keep the program correctness. Thus, we have implemented a reorder buffer and checkpointing mechanisms to enforce the in-order commit of instructions and enable a recovery mechanism in case of exceptions or interrupts.

To enable the execution of non-dependent instructions, we use the register renaming to remove the Write-After-Write hazards. Since the register file is accessed in-order, Write-

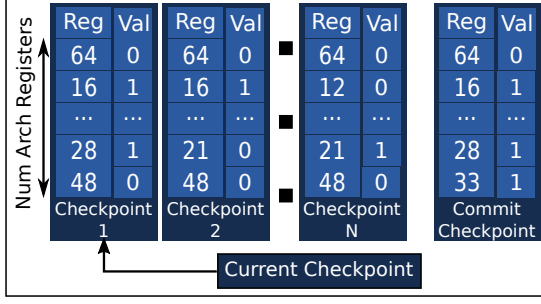


Fig. 2. Block diagram of the renaming table. It has an entry for each architectural register containing the physical register and a ready bit. The structure has one table copy for each potential branch speculated in flight to recover a branch miss-prediction in one cycle. It also contains a copy of the renaming at the commit state for recovering an exception.

After-Read hazards do not affect the correct pipeline execution. We have also extended our design with additional pipelined functional units and a non-blocking memory unit to reduce the issue stalls.

Sargantana’s modular pipeline allows us to integrate new FUs. This feature, along with the out-of-order write-back, eases the integration of custom accelerators for domain-specific applications with long latency instructions as FU.

1) *Register Files and Renaming Mechanism*: Our core uses three different register files for the different types of operands: integer, floating point, and SIMD. These register files have 64 physical registers each to enable the disambiguation of the register renaming mechanism (there are 32 architectural registers). The integer register file has two ports for reading the two possible integer operands. The floating point register file has three different reading ports since the floating point ISA extensions require them. Finally, the SIMD register file has four different read ports. There are two ports for the operand, one port for the old value of the destination register needed for the mask operations, and the mask itself located on the register *v0*.

The register renaming mechanism consists of the rename table and the free list. The rename table (See Figure 2) is a structure that contains 32 entries that map each architectural register to one of the 64 physical registers that stores the most updated value for the architectural register. Additionally, for each entry, there is a ready bit that indicates whether the value of the register is in the register file or must be caught on the fly using the bypass logic. The free list (see Figure 3) is a FIFO queue that stores all the registers that are not being used by any uncommitted instruction. The free list uses a pointer to signal the first free register available and another pointer to signal where a committed register should be written.

The register rename mechanism is triggered on each cycle. Each time the core renames, a new instruction checks where the architectural registers are mapped. If the processed instruction produces a result that must be written on the register file, it takes a free register from the free list and writes its index into the corresponding entry of the architectural destination register.

Since instructions are being speculatively written in the register file, we need a fast one-cycle roll-back mechanism to recover the architectural state in case of exception, interrupt, or branch miss-prediction. For the first two cases, we keep a copy

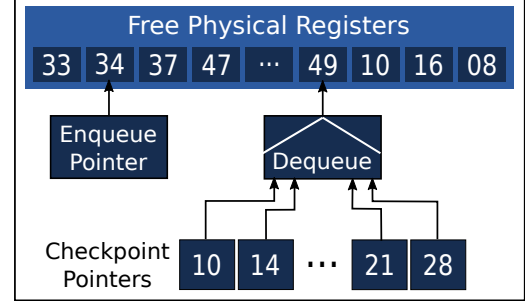


Fig. 3. Block diagram of the free list. It is a FIFO-like structure with an enqueue port for adding new free registers and a dequeue port to read physical destination registers to the rename stage. The dequeue port has several checkpoints for recovering a branch miss-prediction.

of the entire rename table (Commit Checkpoint in Figure 2) that is updated only at the commit stage [22]. In the case of the free list, since it follows a FIFO order and a new register is only queued at commit, we just need to set the dequeue pointer just after the queued one.

In the case of branch miss-predictions, we can have several unresolved branches in-flight. Our solution is to enable another checkpointing mechanism that can be rolled-back in one cycle. Hence, a checkpoint is taken each time a branch instruction is renamed, and we delete the checkpoint or roll-back to the previous state when the branch is resolved.

Therefore, the register rename table has N speculation checkpoints, and the *actual checkpoint* is a pointer to the version of the rename table that is used in this cycle. When a new branch instruction comes to the rename stage, the *actual checkpoint* is incremented, and the new checkpoint pointer is associated with the branch instruction. The previous checkpoint mapping is copied to the new checkpoint table. When a branch instruction is miss-predicted, we use its checkpoint tag to recover the corresponding mapping.

The checkpointing mechanism in the free list consists on multiple versions of the dequeue pointer (Checkpoint Pointers in Figure 2). When a branch miss prediction occurs, we just need to start dequeuing again from that checkpoint.

2) *Reorder Buffer and Commit Stage*: To precisely track exceptions and interrupts, the core needs to treat them in order [23]. In-order cores treat exceptions and interrupts straightforwardly but, in an out-of-order write-back core, we need a special structure to commit the instructions in order. This ordering module is the Reorder Buffer (ROB).

The ROB implementation consists of a circular buffer with two pointers to keep track of the first and the last valid instructions. Each time an instruction is introduced on the ROB, an instruction ID is returned and the last valid instruction pointer is incremented. The instruction carries this ID until it finishes its execution. On write-back, the instruction sends its ID to the ROB to mark the instruction as finished. Since there can be up to four instructions finishing at a given cycle, one for each functional unit, the ROB has four of these ports.

On every cycle, the commit stage checks if the oldest instruction of the ROB is finished. If so, the commit stage dequeues the entry to process it. In case the instruction has an exception, the entire ROB is flushed. A similar mechanism

is used for branch miss-predictions. On write-back, branches notify if branch prediction was correct. In case of miss-prediction, the ROB flushes all the instructions younger than the branch using the branch ID.

Another important task of the ROB is to execute stores and Atomic Memory Operations (AMO) in order to keep sequential program semantics. These instructions wait in the memory pipeline until the same instruction exits the ROB to commit. Thus, blocking the entire memory pipeline. We have alleviated this bottleneck enabling two instructions to commit at the same cycle. This way we reduce the amount of cycles stores and AMOs block the memory pipeline.

3) *Issue, Write-Back, and Forwarding Mechanisms*: Ideally, all three register files must have a small number of write ports to reduce complexity and area. However, our core has several pipelined functional units that can generate one or more results per cycle. Furthermore, Floating-Point Unit (FPU) and SIMD execution paths can potentially write to the integer register file. In order to reduce the number of write ports in the register files, some functional units share the same write port to the integer register file, such as FPU and SIMD or Arithmetic Logic Unit (ALU), Multiplication and Division FUs. While, other units like the memory pipeline have specific write ports.

The FPU and SIMD share a write port because it is infrequent that the core needs to convert one floating point register to an integer and extract one element from the SIMD at the same time. Furthermore, the integration is very simple because the FPU has an output queue that can hold the results while the SIMD is writing to the integer register file.

ALU, Multiplication and Division FUs share a port because the latency on these modules is fixed, and it is easy to detect collisions. The mechanism consists of a shift register of size equal to the longest instruction latency (See Table I). When a new instruction is going to be issued, first, it checks that no other instruction is going to write-back in the same cycle. Then, if no collision is detected, it sets the bit in the position of the instruction latency minus one. This shift register copies the bit n to bit $n - 1$ at every cycle. If the module detects a collision, we delay the instruction one cycle.

However, we have observed during the evaluation of the core that collisions between ALU and Multiplication FUs are very frequent. Therefore, we have created an extra write port for Multiplication FU. The performance of this optimization is discussed in Section IV-B.

All these write buses are snooped by instructions that have dependent source operands. Thanks to the ready bit of the register renaming table, each instruction knows if it needs to capture the operand from one of these buses. The snoop system consists of a comparator that checks if the physical register being written equals one of the source operands.

4) *Functional Units*: As we have already mentioned, Sargantana has several functional units that have independent execution paths. Instruction latency is listed in Table I. We have an Arithmetic Logic Unit (ALU), a Multiplication unit, a Division Unit, Branch Resolve Unit, a FPU and a SIMD.

Not all the multi-cycled FUs are fully pipelined. The division unit uses the long division algorithm, which takes several cy-

TABLE I
INSTRUCTION LATENCIES FOR THE DIFFERENT FUNCTIONAL UNITS

Integer	
Integer Arithmetic & Branch Resolve	1 cycle
32-bit/64-bit Multiplication	2/3 cycles
32-bit/64-bit Division	20/38 cycles
Floating Point	
Floating Point Arithmetic & Multiplication	5 cycles
32-bit Division & Square Root	6 to 15 cycles
64-bit Division & Square Root	6 to 25 cycles
Floating-Point Conversion	5 cycles
Vector Integer	
Integer Arithmetic	1 cycle
Integer Multiplication	3 cycles
Scalar to Vector/Vector to Scalar	1 cycle

cles. Since the cost of pipelining the division unit is exorbitant, we have duplicated the capacity of the FU to process two operations in parallel.

The floating point unit we have used is provided and maintained by the University of Zagreb (initially based in ETHZ's Ariane FPU). All the operations are fully pipelined except for the division and square root. The SIMD unit is explained in Subsection III-B6, and the Memory pipeline is described next.

5) *Memory Pipeline*: The memory FU is the most complex execution path of the design. It is composed of a Load/Store Queue (LSQ), the L1 Data (L1D) cache and the Pending Request Memory Queue (PMRQ). Figure 4 depicts the block diagram of the memory FU. The main component is the L1D cache, which was borrowed from the lowRISC project [21]. This cache is VIPT. Thus, the L1D cache translates the virtual page address in parallel with the access of the tag and data arrays. The main feature of the L1D is its non-blocking design, which can sustain several requests in-flight thanks to a Miss Status Holding Registers (MSHR) table and a store buffer. The data cache configuration used in this design is a 16-entry Data Translation Lookaside Buffer (dTLB), 4-way associativity, 64 sets of 512-bit size, and 2 MSHR.

The L1D cache has a hit latency of three cycles and it is pipelined in three stages:

- **Stage 1**: Receives the request from the core. Metadata and data memories are accessed in parallel with the dTLB.
- **Stage 2**: Memory arrays and dTLB answer. The tag of all ways is compared against the access address.
- **Stage 3**: If there is a hit, the cache answers with a Acknowledge message and in case of a Load with the requested data. The memory access is buffered in case of a miss, and the MSHR receives a new request.

Sargantana is able to send multiple memory operations at the data cache to take profit from the non-blocking cache thanks to LSQ and PMRQ buffers. The access address is computed in the first stage, and the instruction is enqueued on the LSQ, a FIFO buffer. In the second stage, if the data cache can accept a new request (it can be blocked if the MSHR is full or if the instruction is a non-committed store), an instruction from the LSQ is dequeued and sent to the data cache. Since the latency on a data cache hit is three cycles, the instruction information is pipelined in parallel to the cache access. In the

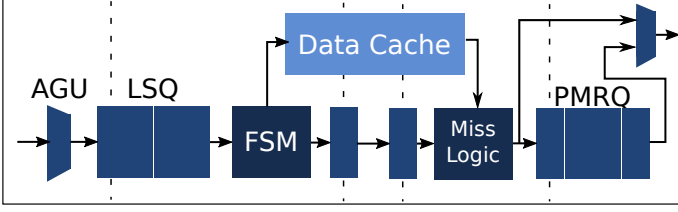


Fig. 4. Block diagram of the memory pipeline. The instruction information of the memory access is pipelined alongside the cache. This information is merged with the cache response. In case of a cache miss, this information is stored on the PMRQ, waiting for future data response.

third cycle, if the cache answers with the request’s response, the instruction is sent to write-back. Otherwise, the instruction is enqueued in the PMRQ to wait for its result. The PMRQ is a Content-Addressable Memory (CAM), so on every L1D cache answer, all entries are checked to wake up any waiting memory requests. The awoken instruction captures the data answer if necessary and advances to the write-back stage.

Load instructions can be issued speculatively to the data cache, but Stores and AMO must be executed in program order. If a Store or AMO arrives at the head of the LSQ, the control logic will wait until the same instruction arrives at the head of the ROB to commit. This restriction delays the execution of younger ready loads that must wait until the execution of the blocking store due to the FIFO nature of the LSQ. Many Out-of-Order cores use memory disambiguation techniques to enable Load to Store overtaking. These techniques rely on checking the accessed address to detect dependencies. However, they require the physical memory addresses to avoid possible Memory Synonyms, and our L1D cache only accepts virtual addresses since it is VIPT.

We have developed an optimization that can perform the Load/Store reordering based on the virtual address. Since the L1D cache is VIPT, we know that the virtual address is used to index the data and metadata memory arrays. Therefore, we already know if two addresses will be mapped to the same set and even which part of the cache line is accessed. Thus, we have upgraded the LSQ placing another FIFO buffer named Store Buffer (SB). When a store arrives at the head of the LSQ buffer and is not ready, instead of blocking younger instruction, it is moved to the SB. When a load arrives at the head of the LSQ, first, it checks that the access address set and offset does not collide with any of the instructions stored in the SB. The performance of this optimization is discussed in Section IV-B

6) *SIMD Unit*: Sargantana is a core designed to execute HPC applications, such as Stencils, Cryptography algorithms, Genomic Analysis algorithms and Machine Learning, among other kernels. Co-design is a key development strategy that brings software specification to the hardware design process. We have applied this co-design strategy to accelerate the execution of some HPC applications. We have observed that a common characteristic of these applications is the richness of data parallelism available. Hence, we have implemented a minimalist SIMD unit of 128 bits. The width is restricted to the L1D bandwidth, which can deliver a maximum data packet of 128 bits. Sargantana supports only a small subset of the RISC-V

TABLE II
SARGANTANA CONFIGURATION FOR THE EXPERIMENTAL SETUP

SoC Configuration & System parameters	
Pipeline	64-bit RISC-V, RV64G variant, 64 physical registers, 128-entry bimodal predictor, 32-entry graduation list
Memory Unit	8-entry LSQ, 8-entry Store Buffer, 16 misses in flight
Data L1	32 KB 4-way set associative, 3-cycle latency, VIPT, 2-entry MSHR
Instruction L1	16 KB 4-way set associative, 2-cycle latency, VIPT
TLBs	Full associative, 16 entries
L2 LLC	512 KBytes, 8-way set associative

Vector extension, including basic arithmetic operations, shifts, comparisons, and vector memory operations. Our SIMD does not support floating point, gather or scatter instructions. The module also offers support for masked operations, with the mask value permanently residing in vector register *v0*.

Another characteristic of our applications is that data types can be reduced to increase the utilization of the SIMD unit. Although the length of the vector registers is fixed, the data element size can be scaled to 8, 16, 32, or 64-bits wide elements. Additionally, we have implemented custom instructions to accelerate specific applications, but we cannot disclose their implementation.

IV. EXPERIMENTAL EVALUATION

A. Experimental Methodology

1) *Experimental Framework*: We have used two frameworks for the evaluation: Verilator RTL simulator for running small bare-metal microkernels, and FPGA emulation on an Alveo U280 for full-system experiments running the applications over a Buildroot Linux distribution (kernel version 5.8.0). Besides, we have taken advantage of the pipeline visualizer Konata for analyzing the impact of the different optimizations. Table II presents the System on Chip (SoC) configuration used.

2) *Experimental Workloads*: We have used RISC-V microbenchmarks during the development of the core to measure the performance of each optimization. We have also used the EEMBC AutoBench [16] and CoreMark [17], a pair of popular benchmarks used to evaluate and compare the performance of cores. Finally, we have evaluated Sargantana with more significant commonly used benchmarks and applications in full system mode for more reliable results.

3) *Academic Processors*: The core designs we have used to compare against Sargantana are Rocket core [18], RISCY [19], and Ariane [8]. They are described in Section II.

4) *Synthesis and Physical Design Environment*: Sargantana has been synthesized in the GlobalFoundries 22nm FD-SOI technology node. We have used Cadence Genus tool v19.11 for the logical synthesis. For extracting the physical metrics of the design, we have used the reports of place and route (PnR) using the Cadence Innovus tool v19.11. The standard cell libraries used have: 8-tracks, low Vt, and a 104 CPP. We have used high-performance single-port SRAMs for the caches.

B. Performance Evaluation Using RISC-V Microbenchmarks

Figure 5 shows the IPC of each of the incremental Sargantana pipeline optimizations done. In this experiment, we take the

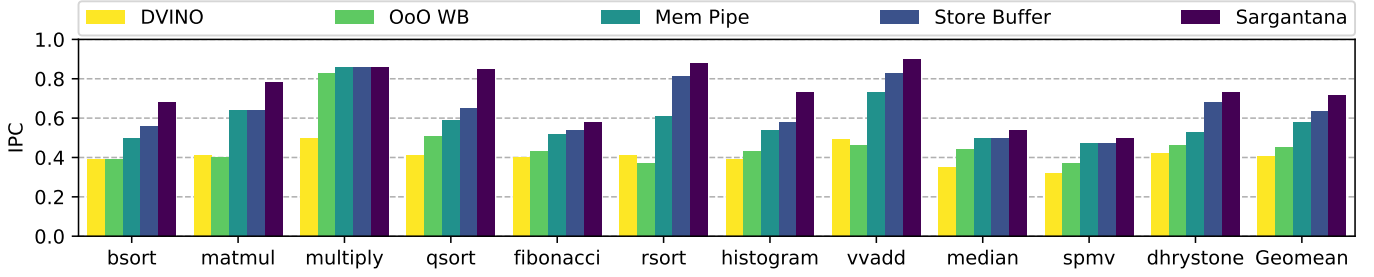


Fig. 5. IPC of RISC-V microbenchmarks for all the incremental optimizations of Sargantana pipeline starting from the DVINO design. The optimizations are the inclusion of the Out-of-Order Write-Back mechanism (OoO WB), the non blocking memory pipeline (Mem Pipe), the integration of the store buffer (Store Buffer), and all the fine-tuning of the different parts of the pipeline (Sargantana). The geometric mean of all the benchmarks is shown on the right bar.

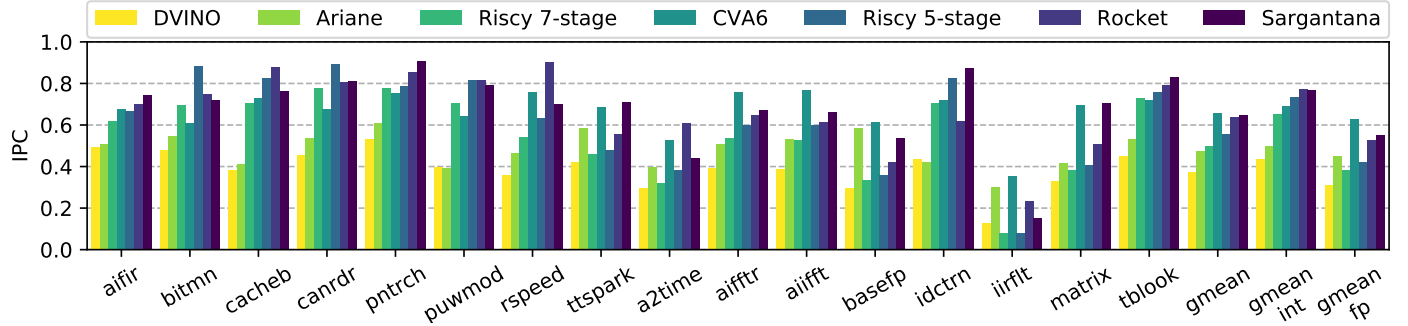


Fig. 6. IPC of the benchmarks of the AutoBench suite for different academic cores. The geometric mean of the integer benchmarks (aifir to ttspark), floating point benchmarks (a2time to tblock) and overall is shown on the right bars.

Sargantana predecessor, "DVINO", as a baseline core. DVINO is a 5-stage in-order core that achieves a low IPC due to its blocking front-end, execution unit and data cache accesses.

The first version of Sargantana (*OoO WB*) only featured the pipelined front-end and the Out-of-Order write-back keeping the blocking memory FU. This first version achieves a $1.11\times$ speed-up by unlocking the execution of non-dependent instructions that were blocked by multi-cycle instructions.

The second version of Sargantana (*Mem Pipe*) features the non-blocking memory pipeline. It enables one memory access per cycle and up to 16 instructions in flight in case of miss. It achieves a speed-up of $1.43\times$ in average compared to the DVINO and $1.48\times$ in both *rsort* and *vvadd* benchmarks. The next optimization (*Store Buffer*) evaluated is the store buffer integrated in the LSQ. This store buffer is especially beneficial on the benchmarks with a huge number of stores like *bsort*, *qsort* or *rsort*. On average, the store buffer gives near a $1.10\times$ more IPC to the design and $1.56\times$ with respect to the DVINO.

The final optimizations on the core are mainly focused on fine-tuning some parts of the pipeline. It reduces the latency on the memory pipeline and increments the number of writing ports of the register file to reduce structural hazard stalls. The fine-tuning of the pipeline increases the IPC by $1.18\times$ and a $1.77\times$ over the original DVINO processor.

C. Performance Comparison Against Other Academic Designs

Figure 6 shows a comparison of the IPC of several academic cores running the EEMBC Autobench benchmark. The geometric mean of all the IPCs of each core is shown at the right of the figure. The IPC comparison on the integer benchmarks shows that Sargantana outperforms all other cores except for the 5-

stage Rocket core. Rocket has a shorter pipeline which achieves a lower branch miss-prediction penalty. However, Sargantana's extra pipeline stages decrease the critical paths and allow the core to run at a higher frequency. Regarding the floating point benchmarks, the IPC is FPU latency dependant. Only CAV6 beats Sargantana on the floating-point benchmarks.

We have also compared Sargantana performance with the CoreMark benchmark (See Figure 7). The figure shows the CoreMark/MHz of each design, which is directly proportional to the IPC. Sargantana gets 2.44 CoreMark/MHz, the higher score of the other state-of-the-art academic in-order cores. Since published results are available for many existing cores, we have compared Sargantana with two industrial reference cores. Sargantana performs better than the Cortex-A5 from Arm, an industrial reference of a single issue, low-power, in-order, 8-stage core. However, it is far from the performance the Cortex-A9, a 8-stage Dual-Issue Out-of-Order core [24].

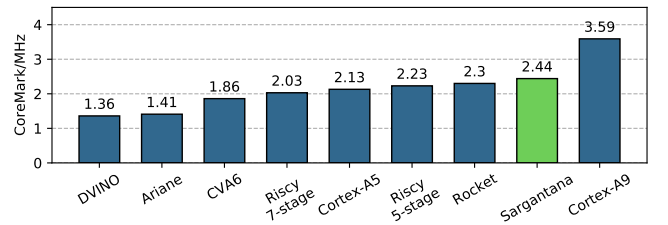


Fig. 7. CoreMark/MHz score of the different cores with the Sargantana highlighted. Sargantana outperforms the other academic in-order designs.

D. Full System Benchmarks Results

We have run commonly used benchmarks and applications on a Linux OS to have more accurate core results in real-

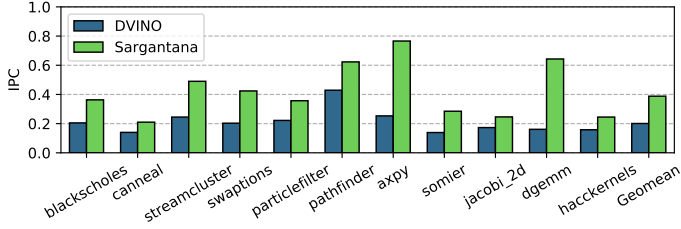


Fig. 8. IPC of DVINO and Sargantana cores running HPC benchmarks running on top of Linux. The geometric mean is shown in the rightmost bar.

world environments. These benchmarks and applications have a much bigger memory footprint that will stress more the memory hierarchy. Figure 8 shows the IPC of Sargantana and its predecessor. Sargantana achieves up to 0.35 and 0.77 GFLOPS (at 1GHz) on *axpy* and *dgemm* respectively. Sargantana outperforms DVINO by a 1.93X.

The big footprint of these benchmarks generates more cache misses than in the microbenchmarks. Sargantana take advantage of the new memory pipeline that can handle several misses on flight. This fact explains that the speed-up on the full system benchmarks is higher than the speed-up on the microbenchmarks used on simulation.

E. Application-Specific Acceleration Using the SIMD Unit

Apart from all the scalar benchmarks executed, Sargantana can accelerate microkernels with the SIMD unit. Taking advantage of the narrow operands of the SIMD, we have accelerated a matrix multiplication and the *compute next* kernel of the Wavefront [14] vectorizing them with different element widths. Apart from these two algorithms, we have accelerated the Classic McEliece decryption algorithm in three different levels of security. We have only used a 32-bit element width since the algorithm can not use the narrower integers. Figure 9 shows the speed-up of the 128-bit SIMD vectorization of the different benchmarks. The WFA and the Matrix multiplication scales with the number of elements inside the vector while the McEliece decryption achieves a constant speed-up of 4.9x.

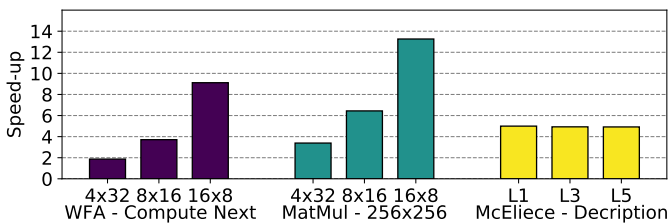


Fig. 9. Speed-up of the vectorized WFA and the matrix multiplication kernels w.r.t scalar version for 32, 16 and 8 bit data elements. At the right is shown the vector speed-up of Classic McEliece kernel for different security levels.

F. Synthesis and Physical Design of Sargantana

The Physical design of Sargantana has been done with 22nm FD-SOI GlobalFoundries technology. Figure 10 shows the floorplan of Sargantana with the pad ring in a 1.9 mm² block. In this technology, the core achieves the frequencies of 0.89 GHz on the slow corner, 1.26 GHz on the typical corner and 1.69 on the fast corner. Doing a deeper analysis of the critical paths of the design, the reports show that the 100

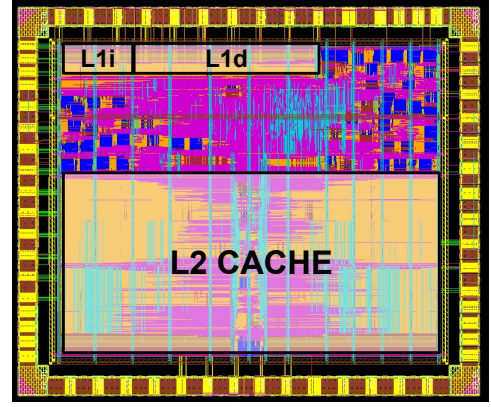


Fig. 10. Floorplan of Sargantana with the pad ring in a 1.9 mm² block in 22nm FD-SOI technology. SRAM macros of cache memories are highlighted.

first critical paths in the design are internal to the FPU. The critical path is difficult to solve since the design is from another institution and it relies on aggressive retiming optimizations done by the synthesis tools that are not under our control.

Besides the frequency results, the PnR reports help us see each module's area consumption on the design. The area post PnR of the design (core with the L1 and L2 caches) is 1.37 mm². Figure 11 contains the area distribution of Sargantana. At the left, there is the area distribution of the entire SoC, while at the right, there is the area distribution of the datapath. The biggest part of the SoC is the L2 cache. Inside the datapath, the module that consumes more area is the execution stage. The execution stage incorporates very large modules. It includes the FPU, the SIMD unit, and the memory pipeline (including large memories for the LSQ and PMRQ).



Fig. 11. Area distribution of the SoC (left side) and the breakdown of the datapath (right side) of Sargantana.

V. ADDITIONAL RELATED WORK

Due to the open-source nature of RISC-V, there has been a remarkable increase in the number of cores and accelerators implementing it in the past years. These implementations range from the most simple in-order energy efficient designs [25] to High-Performance Computing (HPC) oriented out-of-order cores [4], [7]. For improving the performance and the energy efficiency on those HPC workloads, some vector accelerators have been proposed on the recent years [3], [5].

Most open source RISC-V designs are meant to be used by others in their own SoC designs. However, some SoCs and Network on Chip (NoC) [26], [27] designs have been proposed to facilitate the development of new core designs without having to implement all the infrastructure. Moreover, those SoCs

and NoCs open the door to implement much more complex heterogeneous designs incorporating several accelerators.

VI. CONCLUSIONS

In this paper, we have presented Sargantana, a 7-stage in-order core design that features an out-of-order write-back, a non-blocking memory pipeline, and a SIMD unit for accelerating domain-specific applications. We have also presented the Sargantana's ASIC physical design for a state-of-the-art 22nm technology target. The core achieves 0.89, 1.26 and 1.69 GHz on the slow corner, typical corner, and fast corner respectively. This way, Sargantana's maximum frequency matches other academic state-of-the-art processors.

As a result, Sargantana improves by $1.77\times$ the IPC of its predecessor, DVINO (a 5-stage in-order core). Compared to other 6-stage and 7-stage in-order academic cores, Sargantana achieves similar or better IPC in the AutoBench benchmark suite. In the CoreMark benchmark, Sargantana gets 2.44 CoreMark/MHz, the highest score in its class.

Future work will focus on further enhancement of the hardware-software co-design on heterogeneous systems. Such systems will leverage custom cores and accelerators to boost domain-specific applications. We expect those systems to deliver higher performance than current systems, at the expense of much complex architectures that will require a runtime system to ease the programmer workload.

Notwithstanding the predominance of classical architectures, RISC-V will continue to expand its ecosystem, providing a viable alternative for the design of future cores and accelerators. Its openness will allow the exploration and design of novel ISA extensions, eluding prohibitive costs of proprietary ISA licences. Certainly, RISC-V will drive the design of future-generation computing systems, supporting the development of innovative architectures for domain-specific applications.

VII. ACKNOWLEDGEMENTS

This work has been partially supported by the Spanish Ministry of Economy and Competitiveness (contract PID2019-107255GB-C21), by the Generalitat de Catalunya (contract 2017-SGR-1328), by the European Union within the framework of the ERDF of Catalonia 2014-2020 under the DRAC project [001-P-001723], and by Lenovo-BSC Contract-Framework (2020). The Spanish Ministry of Economy, Industry and Competitiveness has partially supported M. Doblas and V. Soria-Pardos through a FPU fellowship no. FPU20-04076 and FPU20-02132 respectively. G. Lopez-Paradis has been supported by the Generalitat de Catalunya through a FI fellowship 2021FI-B00994. S. Marco-Sola was supported by Juan de la Cierva fellowship grant IJC2020-045916-I funded by MCIN/AEI/10.13039/501100011033 and by "European Union NextGenerationEU/PRTR", and M. Moretó through a Ramon y Cajal fellowship no. RYC-2016-21104.,

REFERENCES

- [1] D. Bretthauer, "Open Source Software: A History," 2001.
- [2] A. Waterman and K. Asanović, "The RISC-V Instruction Set Manual Volume I: Unprivileged ISA," tech. rep., SiFive Inc. and EECS Department, University of California, Berkeley, Dec 2019.
- [3] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-ghz+ Scalable and Energy-Efficient RISC-V Vector Processor With Multiprecision Floating-Point Support in 22-nm FD-SOI," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 530–543, 2020.
- [4] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, "SonicBOOM: The 3rd Generation Berkeley Out-of-Order Machine," *4th Workshop on Computer Architecture Research with RISC-V*.
- [5] Y. Lee, A. Waterman, R. Avizienis, H. Cook, C. Sun, V. Stojanović, and K. Asanović, "A 45nm 1.3ghz 16.7 double-precision GFLOPS/W RISC-V processor with vector accelerators," in *40th European Solid State Circuits Conference (ESSCIRC)*, pp. 199–202, 2014.
- [6] J. Abella, C. Bulla, G. Cabo, F. J. Cazorla, A. Cristal, M. Doblas, R. Figueras, A. González, C. Hernández, C. Hernández, V. Jiménez, L. Kosmidis, V. Kostalabros, R. Langarita, N. Leyva, G. López-Paradis, J. Marimon, R. Martínez, J. Mendoza, F. Moll, M. Moretó, J. Pavón, C. Ramírez, M. A. Ramírez, C. Rojas, A. Rubio, A. Ruiz, N. Sonmez, V. Soria, L. Terés, O. Unsal, M. Valero, I. Vargas, L. Villa, and C. Ramirez, "An Academic RISC-V Silicon Implementation Based on Open-Source Components," in *2020 XXXV Conference on Design of Circuits and Integrated Systems (DCIS)*, pp. 1–6, 2020.
- [7] S. Zhang, A. Wright, T. Bourgeat, and A. Arvind, "Composable Building Blocks to Open up Processor Design," in *51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 68–81, 2018.
- [8] F. Zaruba and L. Benini, "The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-bit RISC-V Core in 22-nm FDSOI Technology," *IEEE Transactions on Very Large Scale Integration (VLSI)*, vol. 27, pp. 2629–2640, Nov 2019.
- [9] C. Schmidt, J. Wright, Z. Wang, E. Chang, A. Ou, W. Bae, S. Huang, A. Flynn, B. Richards, K. Asanović, E. Alon, and B. Nikolić, "An Eight-Core 1.44GHz RISC-V Vector Machine in 16nm FinFET," in *IEEE International Solid-State Circuits Conference (ISSCC)*, pp. 58–60, 2021.
- [10] J. Kadamoto, T. Koizumi, A. Fukuda, R. Matsuo, S. Mashimo, A. Fujita, R. Shioya, H. Irie, and S. Sakai, "An Area-Efficient Out-of-Order Soft-Core Processor Without Register Renaming," in *2018 International Conference on Field-Programmable Technology (FPT)*, pp. 374–377, 2018.
- [11] M. Elmohr, M. Saleh, A. Eissa, K. Ahmed, and M. Farag, "Hardware implementation of a SHA-3 application-specific instruction set processor," in *28th International Conference on Microelectronics (ICM)*, pp. 109–112, 2016.
- [12] "RISC-V Vector Extension," 2019.
- [13] G. Myers, "A Fast Bit-Vector Algorithm for Approximate String Matching Based on Dynamic Programming," *J.ACM*, vol. 46, no. 3, p. 395–415, 1999.
- [14] S. Marco-Sola, J. C. Moure, M. Moreto, and A. Espinosa, "Fast gap-affine pairwise alignment using the wavefront algorithm," *Bioinformatics*, vol. 37, pp. 456–463, 09 2020.
- [15] R. J. McEliece, "A Public-Key Cryptosystem Based on Algebraic Coding Theory," *The Deep Space Network Progress Report*, vol. 42-44, pp. 114–116, 1978.
- [16] J. Poovey, T. Conte, M. Levy, and S. Gal-On, "A Benchmark Characterization of the EEMBC Benchmark Suite," *IEEE Micro*, vol. 29, pp. 18–29, 2009.
- [17] S. Gal-On and M. Levy, "Exploring coremark a benchmark maximizing simplicity and efficacy," *The Embedded Microprocessor Benchmark Consortium*, 2012.
- [18] K. Asanović, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz, S. Karandikar, B. Keller, D. Kim, J. Koenig, Y. Lee, E. Love, M. Maas, A. Magyar, H. Mao, M. Moreto, A. Ou, D. A. Patterson, B. Richards, C. Schmidt, S. Twigg, H. Vo, and A. Waterman, "The Rocket Chip Generator," Tech. Rep. UCB/EECS-2016-17, University of California, Berkeley, Apr 2016.
- [19] G. Hills, A. Wright, S. Fuller, M. Bishop, T. Srimani, P. Kanhaiya, R. Ho, Y. Stein, D. Murphy, A. Arvind, A. Chandrakasan, and M. Shulaker, "Modern microprocessor built from complementary carbon nanotube transistors," *Nature*, vol. 572, pp. 595–602, 08 2019.
- [20] T. Yeh and Y. N. Patt, "Two-Level Adaptive Training Branch Prediction," in *24th Annual International Symposium on Microarchitecture, MICRO 24*, p. 51–61, ACM, 1991.
- [21] lowRISC, "lowRISC 64-bit SoC," <https://www.lowrisc.org/>.
- [22] D. Sager, D. P. Group, and I. Corp, "The microarchitecture of the pentium 4 processor," *Intel Technology Journal*, vol. 1, p. 2001, 2001.
- [23] J. Smith and A. Pleszkun, "Implementing precise interrupts in pipelined processors," *IEEE Transactions on Computers*, vol. 37, no. 5, pp. 562–573, 1988.
- [24] C. Celio, D. A. Patterson, and K. Asanović, "The berkeley out-of-order machine (boom): An industry-competitive, synthesizable, parameterized risc-v processor," Tech. Rep. UCB/EECS-2015-167, EECS Department, University of California, Berkeley, Jun 2015.
- [25] M. Gautschi, P. D. Schiavone, A. Traber, I. Loi, A. Pullini, D. Rossi, E. Flamand, F. K. Gürkaynak, and L. Benini, "Near-Threshold RISC-V Core With DSP Extensions for Scalable IoT Endpoint Devices," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 10, pp. 2700–2713, 2017.
- [26] J. Balkind, M. McKeown, Y. Fu, T. Nguyen, Y. Zhou, A. Lavrov, M. Shahrad, A. Fuchs, S. Payne, X. Liang, M. Matl, and D. Wentzlaff, "OpenPiton: An Open Source Manycore Research Framework," in *21th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS'16*, pp. 217–232, 2016.
- [27] N. K. Choudhary, S. V. Wadhavkar, T. A. Shah, H. Mayukh, J. Gandhi, B. H. Dwiel, S. Navada, H. H. Najaf-abadi, and E. Rotenberg, "FabScalar: Composing Synthesizable RTL Designs of Arbitrary Cores within a Canonical Superscalar Template," *SIGARCH Comput. Archit. News*, vol. 39, p. 11–22, jun 2011.